DATA STRUCTURES

CSC-211

Lab Mid Project

Network Packets Analysis System

Group Members

Name	Registration Number
Abdul Saboor Hassan	SP25-BCT-001
Anas Imran	SP25-BCT-005
Muhammad Adeen Khan	SP25-BCT-030

Instructor Name: Mam Najla Raza

Introduction

- With the rapid growth of computer networks and internet usage, the need for efficient monitoring and security analysis of network traffic has become increasingly important. Every data transmission over a network is divided into small units called packets, which may carry sensitive information and travel through multiple layers of network protocols. Ensuring that these packets are properly analyzed is essential for detecting anomalies, preventing unauthorized access, and maintaining network security.

- This project presents a **Packet Analyzer and Intrusion Detection System (IDS)** developed in C++. The system simulates the working of real-world network monitoring tools by processing network packets using multiple data structures such as queues, stacks, arrays, and linked lists. Each packet is analyzed based on its attributes such as source IP, destination IP, protocol type, and payload data.

- The system assigns a **risk score** to each packet using a rule-based mechanism to identify potentially malicious or suspicious activity. Packets with higher risk levels are flagged and stored separately for further inspection, while normal packets are logged for record-keeping. Additionally, suspicious IP addresses are tracked to help identify repeated threats.

- By combining fundamental data structures with networking concepts, this project demonstrates how packet scheduling, protocol analysis, and security monitoring can be implemented in a simplified yet realistic manner. The system provides a strong foundation for understanding how real-world tools like network analyzers and intrusion detection systems operate in cybersecurity environments.

Functionality Description Table

No.	Functionality	Description	Data Structure Used	Networking Concept
1	Packet Generation	Creates a network packet with source IP, destination IP, protocol, priority, and payload (data).	Struct (Packet)	Packet formation in networks
2	Packet Queueing	Stores incoming packets and arranges them based on priority before processing.	Priority Queue (Array-based)	Packet scheduling in routers
3	Packet Dequeue & Processing	Removes highest priority packet for analysis and processing.	Array (Queue implementation)	Router packet forwarding
4	Protocol Analysis	Breaks protocol string into layers (e.g., ETH > IP > TCP > HTTP) and analyzes each layer.	Stack	OSI model layer processing
5	Packet Logging	Stores all processed packets for history and auditing.	Singly Linked List	Network traffic logging
6	Suspicious Packet Logging	Stores packets flagged as suspicious along with detection reasons.	Singly Linked List	Intrusion event recording
7	Suspicious IP Tracking	Maintains a list of IP addresses that generate suspicious packets.	Array	Firewall blacklist system
8	IDS Risk Scoring	Assigns a numerical risk score (0–100) based on packet content and behavior.	Functions + Conditional Logic	Intrusion Detection System (IDS)
9	Threat Classification	Classifies packets as SAFE, MEDIUM, HIGH, or CRITICAL based on risk score.	Conditional Statements	Security threat evaluation
10	Payload Analysis	Checks packet data for malicious keywords like “attack”, “hack”, etc.	Custom String Matching Algorithm	Payload inspection in IDS

Source Code

```
#include <iostream>
#include <string>
using namespace std;
// ========================= PACKET =========================
struct Packet {
    int id;
    string sourceIP;
    string destIP;
    string protocol;
    int priority;
    string data;
};
// ========================= STRING SEARCH =========================
bool contains(string text, string keyword) {
    int n = text.length();
    int m = keyword.length();
    for (int i = 0; i <= n - m; i++) {
        int j = 0;
        while (j < m && text[i + j] == keyword[j]) {
            j++;
        }
        if (j == m)
            return true;
    }
    return false;
```

```
}
// ========================= IDS SCORING SYSTEM ====================
int calculateRisk(Packet p) {
    int score = 0;
    if (p.sourceIP == "0.0.0.0")
        score += 40;
    if (contains(p.protocol, "UNKNOWN"))
        score += 25;
    if (contains(p.data, "attack"))
        score += 30;
    if (contains(p.data, "hack"))
        score += 30;
    if (contains(p.data, "virus"))
        score += 20;
    return score;
}
string classifyRisk(int score) {
    if (score >= 80)
        return "CRITICAL ATTACK";
    else if (score >= 60)
        return "HIGH RISK";
    else if (score >= 30)
        return "MEDIUM RISK";
    else
        return "SAFE";
}
```

```
// ========================= PRIORITY QUEUE =========================
class PacketQueue {
private:
    Packet arr[100];
    int size;
public:
    PacketQueue() { size = 0; }
    bool isEmpty() { return size == 0; }
    bool isFull() { return size == 100; }
    void enqueue(Packet p) {
        if (isFull()) {
            cout << "Queue Overflow! Packet dropped.\n";
            return;
        }
        int i = size - 1;
        while (i >= 0 && arr[i].priority > p.priority) {
            arr[i + 1] = arr[i];
            i--;
        }
        arr[i + 1] = p;
        size++;
    }
    Packet dequeue() {
        if (isEmpty()) return {-1,"","","",0,""};
        Packet p = arr[0];
        for (int i = 0; i < size - 1; i++)
            arr[i] = arr[i + 1];
```

```
        size--;
        return p;
    }
    void display() {
        cout << "\n===== Packet Queue =====\n";
        if (isEmpty()) {
            cout << "Queue empty.\n";
            return;
        }
        for (int i = 0; i < size; i++) {
            cout << arr[i].id << " | "
                 << arr[i].sourceIP << " -> "
                 << arr[i].destIP << " | "
                 << arr[i].protocol << " | "
                 << arr[i].data << " | P:" << arr[i].priority << endl;
        }
    }
};
// ========================= STACK =========================
class ProtocolStack {
private:
    string arr[10];
    int top;
public:
    ProtocolStack() { top = -1; }
    void push(string x) { if (top < 9) arr[++top] = x; }
    string pop() { return (top >= 0) ? arr[top--] : ""; }
```

```
    bool isEmpty() { return top == -1; }
};

// ==================== NORMAL LOGS (LINKED LIST) ==============
class LogNode {
public:
    Packet data;
    LogNode* next;
    LogNode(Packet p) {
        data = p;
        next = NULL;
    }
};
class PacketLog {
private:
    LogNode* head;
public:
    PacketLog() { head = NULL; }
    void add(Packet p) {
        LogNode* nn = new LogNode(p);
        if (!head) head = nn;
        else {
            LogNode* temp = head;
            while (temp->next)
                temp = temp->next;
            temp->next = nn;
        }
```

```
    }
    void display() {
        cout << "\n===== Processed Packet Logs =====\n";
        if (!head) {
            cout << "No logs.\n";
            return;
        }
        LogNode* temp = head;
        while (temp) {
            cout << temp->data.id << " | "
                 << temp->data.sourceIP << " -> "
                 << temp->data.destIP << " | "
                 << temp->data.protocol << " | "
                << temp->data.data << endl;
            temp = temp->next;
        }
    }
};
// ========================= SUSPICIOUS LOG =========================
class SuspiciousNode {
public:
    Packet data;
    string reason;
    SuspiciousNode* next;
    SuspiciousNode(Packet p, string r) {
        data = p;
        reason = r;
```

```
        next = NULL;
    }
};
class SuspiciousLog {
private:
    SuspiciousNode* head;
public:
    SuspiciousLog() { head = NULL; }
    void add(Packet p, string reason) {
        SuspiciousNode* nn = new SuspiciousNode(p, reason);
        if (!head) head = nn;
        else {
            SuspiciousNode* temp = head;
            while (temp->next)
                temp = temp->next;
            temp->next = nn;
        }
    }
    void display() {
        cout << "\n===== Suspicious Packet Logs =====\n";
        if (!head) {
            cout << "No suspicious packets.\n";
            return;
        }
        SuspiciousNode* temp = head;
        while (temp) {
            cout << temp->data.id << " | "
```

```
                << temp->data.sourceIP << " -> "
                << temp->data.destIP
                << " | Risk: " << temp->reason
                << " | Data: " << temp->data.data << endl;
            temp = temp->next;
        }
    }
};
// ========================= SUSPICIOUS Ips =========================
class SuspiciousIPs {
private:
    string ips[100];
    int count;
public:
    SuspiciousIPs() { count = 0; }
    bool exists(string ip) {
        for (int i = 0; i < count; i++)
            if (ips[i] == ip) return true;
        return false;
    }
    void add(string ip) {
        if (!exists(ip))
            ips[count++] = ip;
    }
    void display() {
        cout << "\n===== Suspicious IPs =====\n";
        if (count == 0) {
```

```
            cout << "None detected.\n";
            return;
        }
        for (int i = 0; i < count; i++)
            cout << ips[i] << endl;
    }
};
// ========================= ANALYZER =========================
void analyzeProtocol(string protocol) {
    ProtocolStack st;
    string temp = "";
    for (int i = 0; i < protocol.length(); i++) {
        if (protocol[i] == '>') {
            st.push(temp);
            temp = "";
        } else temp += protocol[i];
    }
    st.push(temp);
    cout << "\n===== Protocol Analysis =====\n";
    while (!st.isEmpty()) {
        string layer = st.pop();
        cout << layer << " -> ";
        if (layer == "ETH")
            cout << "MAC Layer";
        else if (layer == "IP")
            cout << "Routing Layer";
        else if (layer == "TCP")
```

```
            cout << "Reliable Transfer";
        else if (layer == "HTTP")
            cout << "Web Layer";
        else
            cout << "Unknown";
        cout << endl;
    }
}
// ========================= MAIN =========================
int main() {
    PacketQueue queue;
    PacketLog log;
    SuspiciousLog slog;
    SuspiciousIPs sip;
    int choice, id = 1;
    do {
        cout << "\n====== IDS Packet Analyzer ======\n";
        cout << "1. Generate Packet\n";
        cout << "2. View Queue\n";
        cout << "3. Process Packet\n";
        cout << "4. View Logs\n";
        cout << "5. View Suspicious IPs\n";
        cout << "6. View Suspicious Packet Logs\n";
        cout << "7. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;
        if (choice == 1) {
```

```
        Packet p;
        p.id = id++;
        cout << "Source IP: ";
        cin >> p.sourceIP;
        cout << "Dest IP: ";
        cin >> p.destIP;
        cout << "Protocol: ";
        cin >> p.protocol;
        cout << "Priority (1-3): ";
        cin >> p.priority;
        cin.ignore();
        cout << "Data: ";
        getline(cin, p.data);

        queue.enqueue(p);
    }
    else if (choice == 2) {
        queue.display();
    }
    else if (choice == 3) {
        Packet p = queue.dequeue();
        if (p.id != -1) {
            cout << "\nProcessing Packet ID: " << p.id << endl;
            analyzeProtocol(p.protocol);
            log.add(p);
            int risk = calculateRisk(p);
            string level = classifyRisk(risk);
```

```
                cout << "\n--- IDS REPORT ---\n";
                cout << "Risk Score: " << risk << "/100\n";
                cout << "Status: " << level << endl;

                if (risk >= 60) {
                    cout << "ALERT: Threat Detected!\n";
                    sip.add(p.sourceIP);
                    slog.add(p, level);
                }
            }
        }
        else if (choice == 4) {
            log.display();
        }
        else if (choice == 5) {
            sip.display();
        }
        else if (choice == 6) {
            slog.display();
        }
    } while (choice != 7);
    return 0;
}
```

Output

Main Menu

```
====== IDS Packet Analyzer ======
1. Generate Packet
2. View Queue
3. Process Packet
4. View Logs
5. View Suspicious IPs
6. View Suspicious Packet Logs
7. Exit
Enter choice: 
```

Packet Generation Module

```
Enter choice: 1
Source IP: 192.168.1.1
Dest IP: 138.101.2.2
Protocol: tcp>ip>http
Priority (1-3): 1
Data: i study in comsats
```

View Packet Queue for Processing

```
===== Packet Queue =====
1 | 192.168.1.1 -> 138.101.2.2 | tcp>ip>http | i study in comsats | P:1
```

Process Packet Module

```
===== Protocol Analysis =====
http -> Unknown
ip -> Unknown
tcp -> Unknown

--- IDS REPORT ---
Risk Score: 0/100
Status: SAFE
```

View Packet Logs Module

```
===== Processed Packet Logs =====
1 | 192.168.1.1 -> 138.101.2.2 | tcp>ip>http | i study in comsats
```

View Suspicious Ip Addresses

```
Enter choice: 1
Source IP: 192.168.1.1
Dest IP: 192.169.2.1
Protocol: tcp>ip
Priority (1-3): 1
Data: hack virus attack
```

Generating a packet with suspicious payload

```
Processing Packet ID: 2

===== Protocol Analysis =====
ip -> Unknown
tcp -> Unknown

--- IDS REPORT ---
Risk Score: 80/100
Status: CRITICAL ATTACK
ALERT: Threat Detected!
```

The packet was processed and the risk score was greater than 60

```
===== Suspicious IPs =====
192.168.1.1
```

The ip of the packet was displayed in the suspicious ip section

View Suspicious Logs Module

```
Enter choice: 6

===== Suspicious Packet Logs =====
2 | 192.168.1.1 -> 192.169.2.1 | Risk: CRITICAL ATTACK | Data: hack virus attack
```